Practical No: 10

Write a Program to Implement a text classifier using NaiveBayes with scikit-learn.

In [1]:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

def train_text_classifier(X, y):
    # Split the data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

    # Create a CountVectorizer
    vectorizer = CountVectorizer()
    X_train_vectorized = vectorizer.fit_transform(X_train)
    X_test_vectorized = vectorizer.transform(X_test)

    # Train a Naive Bayes classifier
    classifier = MultinomialNB()
    classifier.fit(X_train_vectorized, y_train)

    # Make predictions
    y_pred = classifier.predict(X_test_vectorized)

    # Print classification report
    print(classification_report(y_test, y_pred))
    return vectorizer, classifier

def classify_text(text, vectorizer, classifier):
    text_vectorized = vectorizer.transform([text])
    prediction = classifier.predict(text_vectorized)
    return prediction[0]

# Example usage
X = [
    "I love this movie, it's amazing!",
    "This book is terrible, I couldn't finish it.",
    "The food at this restaurant is delicious.",
    "The service here is awful, I'm never coming back.",
    "What a great experience, highly recommended!",
]
y = ["positive", "negative", "positive", "negative", "positive"]

vectorizer, classifier = train_text_classifier(X, y)

new_text = "The product exceeded my expectations, I'm very satisfied."
prediction = classify_text(new_text, vectorizer, classifier)
print(f"Prediction for '{new_text}': {prediction}")
```

```
              precision    recall  f1-score   support

    negative       0.00      0.00      0.00       1.0
    positive       0.00      0.00      0.00       0.0

    accuracy                           0.00       1.0
   macro avg       0.00      0.00      0.00       1.0
weighted avg       0.00      0.00      0.00       1.0

Prediction for 'The product exceeded my expectations, I'm very satisfied.': positive
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```